Guía de mejora integral de TalentPact

Oct 12, 2026 · @xavier

Resumen

TalentPact ya funciona como marketplace: las empresas ven un pool real y anónimo desde Supabase y, tras pagar, reciben el contacto real del candidato. Los perfiles de demostración siguen visibles y marcados como tales. La plataforma (Netlify + Supabase) es correcta para esta fase, con una condición que ya está cumplida: migrar las funciones al API moderno.

Veredictos

Pregunta	Respuesta	Estado
¿Es Netlify la plataforma correcta?	Sí para el TFM y el piloto, con las funciones en el API moderno (60 s de límite y obligatorio desde julio de 2027). Para escalar, mover la lógica de datos a Supabase Edge Functions	Migración hecha y verificada con el empaquetador oficial de Netlify
¿Está bien estructurado el código?	No lo estaba: un `index.html` de 858 KB con 5.700 líneas de JavaScript en línea	Reestructurado: 118 KB de HTML, 16 scripts por dominio, CSS aparte, funciones en capas
¿Funciona todo?	98 de 98 tests en verde y todos los paneles probados en el navegador sin errores. Se han encontrado y corregido 9 fallos nuevos, 3 de ellos graves	Ver auditoría
¿Qué falta para ser un producto de selección real?	Pipeline por oferta, notificaciones y MFA (pool y contacto reales ya hechos)	Hoja de ruta al final

El documento Cómo llevar TalentPact al siguiente nivel recoge lo hecho antes en integridad, corrector y condiciones de empleo; esta guía lo integra en un plan completo.

Auditoría de fallos

Se han revisado las 14 funciones, los 16 scripts de cliente, las páginas públicas, la configuración y el repositorio. De 20 problemas, 14 están corregidos y 6 pendientes, todos de gravedad media o baja. El pool y el contacto reales ya están construidos.

Fallo	Impacto	Gravedad	Estado
Tras pagar, la empresa recibía un contacto ficticio: no había pool ni contacto reales en el servidor (resuelto con get-talent-pool y get-contact)	Con claves de Stripe reales se cobrarían 49 € por datos inventados	Alta	Corregido
Cinco funciones aceptaban un `userId` sin token	Leer datos de otro candidato o inyectarle notas	Alta	Corregido
El navegador escribía el prompt del corrector	El candidato podía reescribir su propio examen	Alta	Corregido
Funciones en modo Lambda, obsoleto	Netlify no aceptará despliegues desde el 1 de julio de 2027; límite antiguo de 10 s frente a ~17 s por corrección	Alta	Corregido
El chat de soporte era un proxy abierto (prompt enviado por el cliente)	Cualquiera usaba la clave de Anthropic para lo que quisiera	Alta	Corregido
El servidor de demo servía `.git` (y `.env`), se caía con una URL rara y escuchaba en toda la red	Fuga de claves y caída en directo	Alta	Corregido
El chat inventaba un take-rate del 10-12 %; la FAQ prometía 100 € por reto y un email que no existe	Promesas comerciales falsas ante usuarios	Media	Corregido
Contraseña del superadmin escrita en el JavaScript	Cualquiera la lee; da apariencia de seguridad sin darla	Media	Corregido
Títulos y descripciones de ofertas sin escapar	XSS almacenado en cuanto las ofertas vivan en la base de datos	Media	Corregido

Fallo	Impacto	Gravedad	Estado
`index.html` de 858 KB con todo el código dentro	Imposible de revisar o cachear por partes	Media	Corregido
Cronómetro reiniciable, nota peor que sobrescribía, feedback sin escapar	Trampa fácil y notas que bajan contra lo prometido	Media	Corregido
Publicar oferta perdía salario y modalidad; `#NaN` en el pool; barra de perfil falsa	Datos perdidos y errores visibles	Media	Corregido
Métricas de relleno: "Top 30 %" por tramos, candidatos por oferta y todo el panel de superadmin	Cifras que parecen reales y no lo son	Media	Pendiente
Los retos de audio y vídeo se puntúan por haber grabado, no por el contenido	Nota sin validez en esos retos	Media	Pendiente
El historial del chat podía empezar por el asistente	La API lo rechaza y el chat cae a respuestas fijas	Baja	Corregido
README y comentarios afirmaban que Netlify corta a 10 s	Decisiones de diseño con un dato desfasado	Baja	Corregido
`TalentPact_Pitch_Deck.pptx` y `Talentpact_Pitch_Deck.pptx` solo difieren en mayúsculas	En macOS y Windows uno pisa al otro al clonar	Baja	Pendiente
`.DS_Store` versionado aunque está en `.gitignore`	Ruido en cada commit	Baja	Pendiente
`deno.lock` en un proyecto Node	Confunde sobre el runtime	Baja	Pendiente
Sin versión de Node fijada	Un cambio de versión por defecto en Netlify puede romper el build	Baja	Pendiente

Arreglo de los cuatro pendientes de repositorio, en una sola tanda: borrar uno de los dos `.pptx`, `git rm --cached .DS_Store netlify/.DS_Store`, borrar `deno.lock` y añadir `"engines": { "node": ">=20" }` en `package.json`.

¿Es Netlify la plataforma correcta?

Sí, con las funciones en el API moderno, que ya es como están. Lo que no era correcto era el modo en que se usaba: las 14 funciones estaban escritas en el modo Lambda (`exports.handler`), que Netlify ha declarado obsoleto y dejará de aceptar en despliegues a partir del 1 de julio de 2027 (Netlify, Lambda compatibility).

Plataforma	Límite por petición	Encaje con TalentPact	Coste de cambiar
Netlify, API moderno (actual)	60 s síncrono, 15 min en segundo plano (configuración)	Bueno: web estática + funciones; cabe una corrección de ~17 s	Hecho
Netlify, modo Lambda (anterior)	10 s según el propio código y los foros; 26 s solo pidiéndolo a soporte en plan Pro	Malo: obsoleto y justo para la IA	—
Vercel	300 s por defecto en Hobby y Pro, hasta 800 s en Pro (duración)	Bueno, sin ventaja clara frente a Netlify moderno	Medio: reescribir entradas y configuración
Supabase Edge Functions	150 s de espera, 2 s de CPU por petición (la espera de red no cuenta), 256 MB (límites)	Muy bueno para las funciones de datos: viven junto a la base y a sus reglas RLS	Medio: Deno en vez de Node

Las fuentes de Netlify no coinciden del todo: la página de configuración da 60 s y la de resumen menciona 30 s. Incluso con 30 s la corrección cabe.

Recomendación

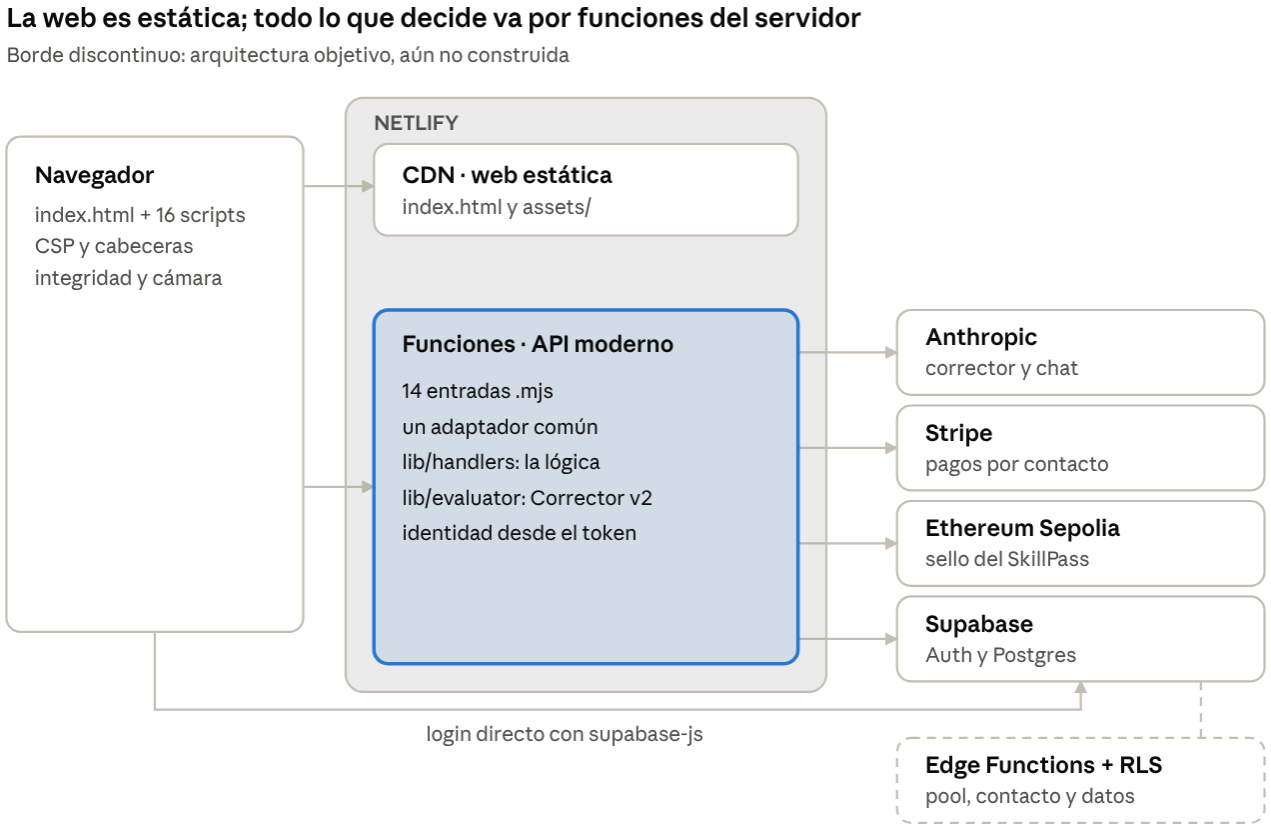
1. **Ahora:** seguir en Netlify con el API moderno. Cambiar de proveedor antes de la defensa no aporta nada.
2. **Cuando haya pool real:** mover las funciones de datos (`save-*`, `get-*`, `delete-account`, el futuro pool y contacto) a Supabase Edge Functions o a consultas directas con RLS. Hoy todas usan la clave de servicio, que se salta las reglas de la base de datos; con RLS la base protege los datos aunque una función tenga un fallo.
3. **Mantener en Netlify** la web, el corrector, los pagos y el SkillPass.

Cómo se ha verificado la migración

- Empaquetado con la CLI oficial de Netlify (`zip-it-and-ship-it` 15.3.4): las 14 funciones salen con `runtimeAPIVersion: 2` .
- Cada zip se ha ejecutado aislado, fuera del repositorio, como en producción: `verify-credential` calcula el hash con ethers, `stripe-webhook` carga Stripe y valida la firma, y `save-profile` responde 401 sin sesión.
- Esa prueba detectó un fallo que habría llegado a producción: en el API moderno, Netlify no incluye en el zip un paquete que solo se carga con un `require` perezoso, así que SkillPass y los pagos habrían fallado con "Cannot find module". Se corrigió importando `ethers` y `stripe` en las entradas que los usan.
- `serve-demo.js` ejecuta ahora las mismas entradas `.mjs` y el mismo adaptador que Netlify.

Arquitectura y estructura del código

El código ya está organizado por capas y por dominio. `index.html` pasa de 858 KB a 118 KB y solo contiene marcado; cada fichero de JavaScript tiene una responsabilidad y un número que fija su orden de carga.



arquitectura · actual y objetivo

El navegador solo habla directamente con Supabase para iniciar sesión; notas, pagos, sellos y datos pasan por funciones que deciden con el token, no con lo que diga el cliente.

```
index.html                    marcado (118 KB)
assets/
  css/app.css                 estilos de la aplicación
  css/modulos.css             integridad, preferencias, filtros
  js/head/                    scroll y analítica con consentimiento
  js/app/01…15-*.js           16 scripts por dominio (ver
assets/js/README.md)
  js/app/99-arranque.js       restaura la sesión cuando todo está cargado
netlify/functions/
  *.mjs                       14 entradas del API moderno (Request →
Response)
  lib/adaptador.mjs           traduce Request/Response en un solo sitio
  lib/handlers/*.js           la lógica de cada función (la que prueban los
tests)
  lib/tp.js                   Supabase, auth, blockchain, Stripe
  lib/evaluator.js            Corrector v2: reglas, filtro de inyección,
recibos
  lib/support-prompt.js       hechos del producto para el chat (desde
cifras_canonicas)
tests/                        98 tests en 11 ficheros, sin red ni claves
```

Cómo se ha hecho sin romper nada. Los 16 scripts son el mismo código cortado por secciones, sin reescribir lógica. El riesgo de trocear scripts clásicos es que una función llamada al cargar quede definida en un fichero posterior. Un análisis del árbol sintáctico (acorn) recorrió todo lo que se ejecuta al cargar, de forma transitiva: encontró 9 referencias en el arranque de sesión, que pasó a `99-arranque.js`, y 0 después. Todos los paneles se han recorrido en el navegador sin errores.

Siguiente paso de estructura (cuando el TFM esté entregado)

1. Pasar a módulos ES con Vite: imports explícitos en vez de ámbito global, y un build que minimiza y versiona los ficheros solo.
2. Quitar los `onclick` del HTML y usar `addEventListener`: permite eliminar `'unsafe-inline'` de la CSP.
3. Sacar los datos (catálogo de retos, 325 KB en `07-retos-comunes.js`) a la base de datos: es también el requisito para que el corrector no dependa del enunciado que envía el navegador.
4. TypeScript en `netlify/functions` y en el cliente, empezando por los tipos de evaluación, recibo y oferta.

Selección de personal de punta a punta

Hoy el producto cubre bien la evaluación y se corta justo después del pago: no hay contacto real, ni seguimiento del candidato por oferta, ni cierre del proceso. Un proceso profesional tiene ocho etapas y cada una deja un rastro que la siguiente usa.

Etapa	Hoy	Mejora	Cómo
1. Oferta	Condiciones (jornada, modalidad, contrato, salario) y skills requeridas; vive en el navegador	Oferta en la base de datos, con número de plazas, fecha de cierre y retos asignados	Tabla `jobs` con RLS por empresa; `publishJob` guarda vía función con token
2. Atracción	Encaje por skills y condiciones con filtros	Nota de encaje ponderada (60 % skills, 25 % condiciones, 15 % disponibilidad) y alertas de ofertas nuevas	Cálculo en servidor; email semanal con las ofertas que encajan
3. Evaluación	Corrector v2, integridad y cámara	Retos del catálogo del servidor, datos distintos por intento y transcripción de audio y vídeo	Tabla `challenges` versionada; Whisper o similar para las grabaciones
4. Preselección	Pool local con filtros y riesgo de integridad	Pool real y anónimo desde Supabase, solo con notas verificadas, y bandeja de revisión humana	Función `get-talent-pool` (alias, skills, condiciones, integridad; nunca datos personales)
5. Desbloqueo	Pago real con Stripe, contacto ficticio	Contacto real entregado solo tras pago confirmado	Función `get-contact`: comprueba `unlocks.status = 'paid'` para esa empresa y ese candidato y devuelve nombre, email, teléfono
6. Entrevista	No existe	La empresa propone huecos al desbloquear; el candidato acepta	Integración con Cal.com o Google Calendar

Etapa	Hoy	Mejora	Cómo
7. Decisión	No existe	Estados por candidato y oferta: preseleccionado, entrevista, oferta, contratado o descartado, con motivo	Tabla `applications` con historial; el motivo de descarte es obligatorio (trazabilidad del AI Act)
8. Cierre	No existe	Feedback al candidato descartado y métricas del proceso para la empresa	Email automático con el feedback de los retos; panel con tiempo hasta contratar y conversión por etapa

Las dos piezas que convierten la demo en marketplace ya están hechas (falta ejecutar `tfm/tech/supabase_schema_pool.sql` en Supabase):

1. `get-talent-pool`: devuelve candidatos reales con su alias anónimo, sus mejores notas verificadas por skill, sus condiciones de empleo y su riesgo de integridad. Requiere la vista `talent_pool` en Supabase, que expone solo columnas anónimas.
2. `get-contact`: la empresa autenticada pide el contacto de un candidato; el servidor comprueba que hay un desbloqueo pagado de esa empresa para ese candidato y solo entonces lo devuelve. El candidato recibe un aviso ("una empresa ha desbloqueado tu perfil").

Cómo funcionan ya en la web: al abrir el panel de empresa con sesión se carga el pool real, cada candidato lleva la marca Real o Demo, "Ver candidatos" de una oferta ordena por encaje (70 % skills requeridas, 30 % jornada y modalidad) y, tras el pago, la ficha muestra el contacto real con la fecha del desbloqueo. Las preferencias de empleo y la visibilidad del candidato se guardan en su perfil de Supabase, que es de donde las lee el pool. Cubierto por `tests/pool.test.js` (5 tests).

Evaluación y corrector

La nota ya es difícil de manipular (Corrector v2: reglas en el servidor, filtro de inyección y recibos firmados). Lo que falta es demostrar que la nota **predice** algo: un producto de selección se juzga por si las personas con buena nota rinden bien en el puesto.

Dimensión	Qué medir	Objetivo	Cómo
Fiabilidad	Misma respuesta, misma nota (test-retest)	Diferencia media ⩽ 3 puntos	Banco de pruebas en CI en cada cambio de prompt o modelo

Dimensión	Qué medir	Objetivo	Cómo
Acuerdo con expertos	κ cuadrática frente a correctores humanos	⩾ 0,80	Ampliar el gold set de 12 a 100 ítems por familia de retos
Validez predictiva	Correlación entre nota y valoración del candidato a los 3 y 6 meses	Positiva y estable	Encuesta a la empresa tras contratar (etapa 8 del proceso)
Equidad	Diferencia de nota por género, edad o lengua materna con respuestas equivalentes	Sin diferencia significativa	Pares de respuestas idénticas con señales demográficas distintas
Robustez	Tasa de bloqueo de ataques y falsas alarmas	100 % de ataques del corpus, 0 falsas alarmas	Ampliar los 3 ataques del gold set a 30 variantes
Revisión humana	Notas limitadas, bloqueadas o con riesgo alto revisadas	100 % en menos de 48 h	Bandeja de revisión con motivo y rastro de cambios

Mejoras del corrector, por orden

1. Catálogo de retos y rúbricas en el servidor (el cliente solo envía el id del reto).
2. Sellar en el SkillPass solo evaluaciones verificadas.
3. Transcribir audio y vídeo antes de corregir.
4. Doble corrector en la zona de duda (45-55 puntos o a ±5 de un umbral): segundo modelo y, si difieren más de 15 puntos, revisión humana.
5. Rúbricas por puesto revisadas por un profesional del área, con respuestas ancla por nivel.
6. Derecho a revisión: botón "Pedir revisión de mi nota" para el candidato.

Seguridad, privacidad y cumplimiento

Las capas básicas están puestas: identidad siempre desde el token, cabeceras de seguridad, política de contraseñas, bloqueo por intentos, cierre por inactividad, corrector y chat sin prompts del cliente, y backoffice real solo con rol de servidor. Lo que queda es lo que exige tener datos reales de personas.

Área	Pendiente	Por qué importa
Acceso	MFA (TOTP de Supabase) obligatorio para empresas y para el rol admin	La empresa ve datos personales desbloqueados
Base de datos	Pasar de la clave de servicio a RLS en las funciones de datos; `npm run check:rls` en CI	Con RLS, un fallo en una función no expone datos de otros
Datos personales	Cifrar teléfono y LinkedIn en reposo; registro de quién consulta cada contacto desbloqueado	Minimización y trazabilidad (RGPD)
Frecuencia	Límite global por usuario e IP (Upstash Redis o el rate limiting de Netlify)	El límite actual es por instancia y no frena un ataque distribuido
Navegador	Quitar `'unsafe-inline'` de la CSP	Cierra la puerta a casi cualquier XSS residual
CORS	Restringir al dominio propio en vez de `*`	Defensa en profundidad
Secretos	Rotar `EVAL_SIGNING_SECRET` y la clave del emisor; no más de 4 KB de variables por función	Higiene operativa
AI Act	Documentación técnica del sistema de alto riesgo, registro de evaluaciones, supervisión humana visible	La selección de personal con IA es de alto riesgo
RGPD	EIPD del tratamiento con cámara y de la evaluación automática; `legal.html` actualizado con cámara, integridad y derecho a revisión	Base legal clara antes de tratar datos reales

La cámara solo comprueba presencia y nunca infiere emociones: el AI Act prohíbe el reconocimiento de emociones en el trabajo. Esta línea no debe cruzarse en ninguna mejora futura de la supervisión.

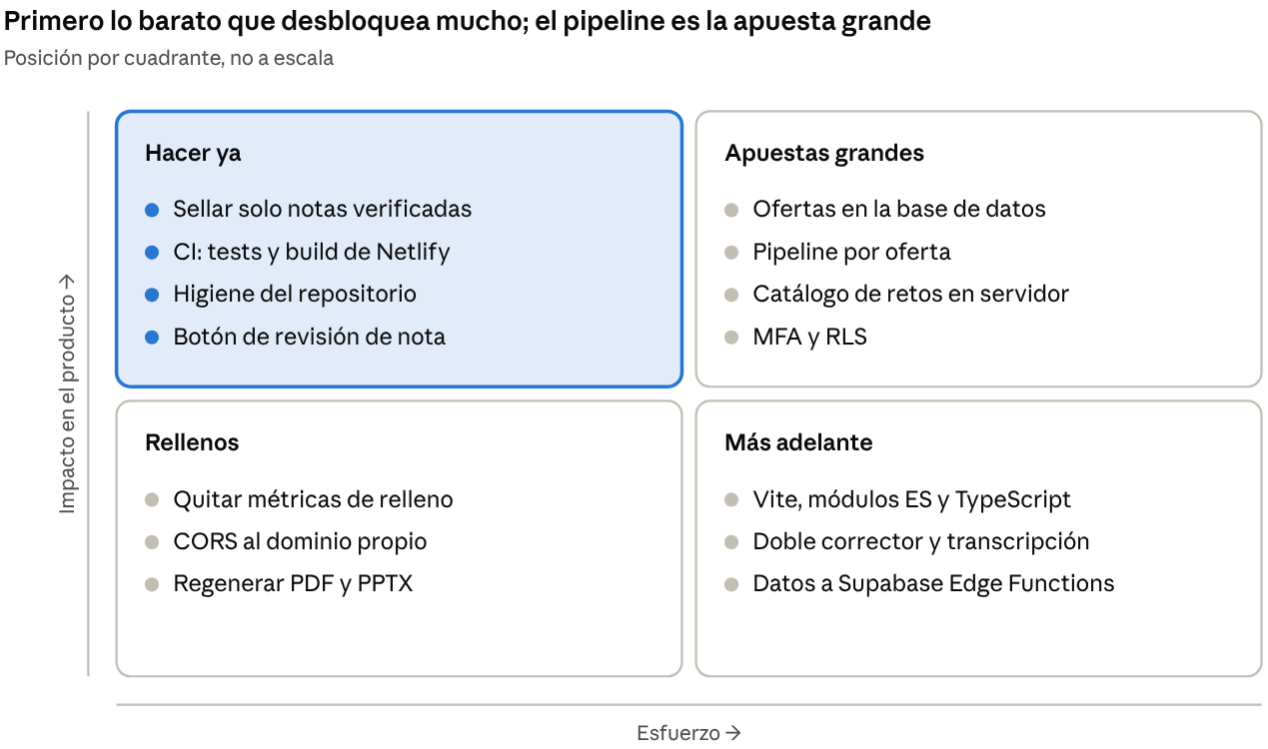
Calidad de ingeniería

Hay una buena base: 98 tests sin red ni claves, un banco de métricas del corrector y un test que obliga a que las cifras de la memoria sean ciertas. Falta automatizarlo y ver qué pasa en producción.

1. **Integración continua.** GitHub Actions con `npm test` en cada push y bloqueo del merge si falla. Añadir `netlify functions:build` al flujo: es la prueba que encontró que `ethers` y `stripe` no viajaban en el zip.
2. **Tests de navegador.** Playwright con tres recorridos: reto completo como invitado, publicar oferta y filtrar el pool, pago en modo test de Stripe. Hoy esos recorridos se prueban a mano.
3. **Entornos.** Netlify Deploy Previews por rama, un proyecto Supabase de staging separado del de producción y claves de Stripe de test fuera de producción.
4. **Observabilidad.** Sentry en cliente y funciones; un panel con errores por función, latencia del corrector y coste de IA por día.
5. **Versiones.** Fijar Node (`engines` en `package.json`) y revisar la cascada de modelos del corrector y del chat contra los modelos vigentes, recalibrando con el banco de pruebas en cada cambio.
6. **Higiene del repositorio.** Los cuatro pendientes de la auditoría (`.pptx` duplicado, `.DS_Store` , `deno.lock` , versión de Node) y sacar los PDF y PPTX generados a una release en vez de versionarlos.
7. **Regenerar entregables.** Los PDF, DOCX y PPTX de `entrega_final/` citan 84 tests; los `.md` y `.html` ya dicen 98.

Hoja de ruta priorizada

El orden: primero lo barato que deja la base fiable, después la apuesta que convierte la demo en marketplace, y lo demás cuando haya usuarios reales.



prioridades · impacto × esfuerzo

Orden de ejecución

1. **Antes de la defensa** (1 semana): higiene del repositorio, regenerar PDF y PPTX, ejecutar `supabase_schema_integrity.sql`, definir `EVAL_SIGNING_SECRET` y desplegar web y funciones juntas.
2. **Base fiable** (2-3 semanas): CI con tests y `netlify functions:build`, sellar solo notas verificadas, botón de revisión y quitar métricas de relleno.
3. **Marketplace real** (4-6 semanas): ofertas en la base de datos y pipeline por oferta con estados y motivos. El pool y el contacto reales ya están hechos.
4. **Blindaje** (2-3 semanas): MFA, RLS en las funciones de datos, límite global de frecuencia y CSP sin `'unsafe-inline'`.
5. **Escala** (continuo): catálogo de retos en servidor, transcripción, doble corrector, validez predictiva y módulos ES con TypeScript.

Las semanas son una estimación para un equipo de dos personas, no un compromiso.